

week07报告

所有相关文件可以在 [这里](#) 找到。

所有图片中的内容，包括代码、日志、控制台输出，均可以在该文档中找到。

一、需求实现情况

1. 实体类 `Student` 实现

- 字段完整：包含 `id`、`name`、`score`、`major`，均为 `private` 且提供 `getter` 方法。
- 正确重写 `hashCode` 和 `equals`，以 `id` 为唯一标识符，确保去重逻辑有效。
- `toString` 方法格式化输出学生信息，符合要求。

```
import java.util.*;

public class Student{
    private final int id;
    private final String name;
    private double score;
    private final String major;

    public Student(int id, String name, double score, String major) {
        this.id = id;
        this.name = name;
        this.score = score;
        this.major = major;
    }

    public int getId() { return id; }
    public String getName() { return name; }
    public double getScore() { return score; }
    public String getMajor() { return major; }

    public void setScore(double score) { this.score = score; }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (o.getClass() != this.getClass()){
            return false;
        }
        Student student = (Student) o;
        return id == student.id;
    }

    @Override
    public int hashCode() { return Objects.hash(id); }

    @Override
    public String toString() {
        return String.format("Student{id=%d, name='%s', score=%.1f, major='%s'",
```

```
        id, name, score, major);
    }
}
```

2. 程序逻辑实现

- **存储与打印**: 使用 `List` 存储初始数据并打印, 输出完整。
- **去重筛选**: 通过 `HashSet` 去重, 筛选成绩 ≥ 60 的学生。去重逻辑正确, 但初始数据中 `id=2` 的两个学生因成绩均未及格, 未出现在及格列表。
- **分组统计**: 通过 `HashMap` 按专业分组, 但需手动预置专业列表, 灵活性不足。分组后打印学生列表及平均成绩, 计算正确。
- **排序**: 使用自定义 `StudentComparator` 实现成绩降序、同名按字母升序, 逻辑正确。
- **查找与更新**: 通过 `HashMap` 按 `id` 查找, 更新成绩为95分。代码中 `id=5` 的学生更新后, 全局统计中正确反映新成绩。
- **全局统计**: 通过 `Collections.min` 结合逆序 `Comparator` 获取最高分学生, 逻辑巧妙。统计及格与不及格人数, 数据准确。

```
import java.util.*;

public class StudentComparator implements Comparator<Student>{
    @Override
    public int compare(Student s1, Student s2) {
        if(s1.getScore()==s2.getScore()){
            return s1.getName().compareTo(s2.getName());
        }else{
            return (int)(s2.getScore()-s1.getScore());
        }
    }
}
```

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        // 初始化数据
        List<Student> students = Arrays.asList(
            new Student(1, "Alice", 85.0, "Computer Science"),
            new Student(2, "Bob", 45.0, "Mathematics"),
            new Student(3, "Charlie", 78.5, "Physics"),
            new Student(4, "David", 59.0, "Computer Science"),
            new Student(5, "Eve", 93.0, "Mathematics"),
            new Student(6, "Frank", 70.0, "Physics"),
            new Student(2, "Duplicate Bob", 45.0, "Mathematics"),
            new Student(7, "Grace", 88.5, "Computer Science")
        );
    }
}
```

```

// 步骤1: 存储并打印初始数据
System.out.println("Initial List of Students:");
students.forEach(System.out::println);

// 步骤2: 去重并筛选及格学生
HashSet<Student> uniqueStudents = new HashSet<>(students);
HashSet<Student> passed = new HashSet<>();
for (Student s : uniqueStudents) {
    if (s.getScore() >= 60) passed.add(s);
}
System.out.println("\nPassed Students (After Removing Duplicates):");
passed.forEach(System.out::println);

// 步骤3: 按专业分组
HashMap<String, List<Student>> grouped = new HashMap<>();
grouped.put("Computer Science", new ArrayList<Student>());
grouped.put("Mathematics", new ArrayList<Student>());
grouped.put("Physics", new ArrayList<Student>());
for (Student s : uniqueStudents) {
    grouped.get(s.getMajor()).add(s);
}
System.out.println("\nStudents Grouped by Major:");
for (Map.Entry<String, List<Student>> entry : grouped.entrySet()){
    System.out.println("Major: " + entry.getKey());
    for (Student i : entry.getValue()){
        System.out.println(i.toString());
    }
}

// 步骤4: 计算平均成绩
System.out.print("\nAverage Score by Major:");
for (Map.Entry<String, List<Student>> entry : grouped.entrySet()){
    System.out.print("\nMajor: " + entry.getKey() + ", Average Score: ");
    double total = 0;
    for (Student i : entry.getValue()){
        total += i.getScore();
    }
    System.out.print(total/entry.getValue().size());
}

// 步骤5: 排序
List<Student> sorted = new ArrayList<>(uniqueStudents);
sorted.sort(new StudentComparator());
System.out.println("\n\nStudents Sorted by Score (Descending):");
sorted.forEach(System.out::println);

// 步骤6: 查找与更新
Map<Integer, Student> map = new HashMap<>();
for (Student s : uniqueStudents){
    map.put(s.getId(), s);
}
int[] searchIds = {5, 100};
System.out.println("\nUpdated consequence:");

```

```

        for (int id : searchIds) {
            if (map.containsKey(id)) {
                Student s = map.get(id);
                s.setScore(95);
                System.out.println("Updated: " + s);
            } else {
                System.out.println("Student with ID " + id + " not found.");
            }
        }
    }

    // 步骤7: 全局统计
    // 要实现降序最简单的方法就是颠倒大小比较, 所以这里变成了min
    // 🤔👉👈
    Student max = Collections.min(uniqueStudents, new StudentComparator());
    int passedCount = 0;
    for (Student s : uniqueStudents){
        if(s.getScore() >= 60){
            passedCount += 1 ;
        }
    }
    System.out.println("\nGlobal Statistics:");
    System.out.println("Highest Scoring Student: " + max);
    System.out.println("Number of Passed Students: " + passedCount);
    System.out.println("Number of Failed Students: " + (uniqueStudents.size() -
passedCount));
    }
}

```

二、输出验证

- **初始数据:** 包含所有学生, 包括重复 `id=2` 的记录。
- **及格学生:** 过滤后仅包含成绩 ≥ 60 且 `id` 唯一的学生 (如 Alice、Eve 等)。
- **分组与平均分:** 例如, Computer Science 组平均分计算为 $(85.0 + 59.0 + 88.5)/3 = 77.5$, 输出正确。
- **排序结果:** Eve 更新后成绩为95分, 排在首位; 成绩相同者按名字升序排列。
- **查找与更新:** `id=5` 更新成功, `id=100` 提示未找到。
- **全局统计:** 最高分为更新后的 Eve (95分), 及格人数统计正确。

Initial List of Students:

```

Student{id=1, name='Alice', score=85.0, major='Computer Science'}
Student{id=2, name='Bob', score=45.0, major='Mathematics'}
Student{id=3, name='Charlie', score=78.5, major='Physics'}
Student{id=4, name='David', score=59.0, major='Computer Science'}
Student{id=5, name='Eve', score=93.0, major='Mathematics'}
Student{id=6, name='Frank', score=70.0, major='Physics'}
Student{id=2, name='Duplicate Bob', score=45.0, major='Mathematics'}
Student{id=7, name='Grace', score=88.5, major='Computer Science'}

```

Passed Students (After Removing Duplicates):

```

Student{id=1, name='Alice', score=85.0, major='Computer Science'}
Student{id=3, name='Charlie', score=78.5, major='Physics'}

```

```
Student{id=5, name='Eve', score=93.0, major='Mathematics'}
Student{id=6, name='Frank', score=70.0, major='Physics'}
Student{id=7, name='Grace', score=88.5, major='Computer Science'}

Students Grouped by Major:
Major: Computer Science
Student{id=1, name='Alice', score=85.0, major='Computer Science'}
Student{id=4, name='David', score=59.0, major='Computer Science'}
Student{id=7, name='Grace', score=88.5, major='Computer Science'}
Major: Mathematics
Student{id=2, name='Bob', score=45.0, major='Mathematics'}
Student{id=5, name='Eve', score=93.0, major='Mathematics'}
Major: Physics
Student{id=3, name='Charlie', score=78.5, major='Physics'}
Student{id=6, name='Frank', score=70.0, major='Physics'}

Average Score by Major:
Major: Computer Science, Average Score: 77.5
Major: Mathematics, Average Score: 69.0
Major: Physics, Average Score: 74.25

Students Sorted by Score (Descending):
Student{id=5, name='Eve', score=93.0, major='Mathematics'}
Student{id=7, name='Grace', score=88.5, major='Computer Science'}
Student{id=1, name='Alice', score=85.0, major='Computer Science'}
Student{id=3, name='Charlie', score=78.5, major='Physics'}
Student{id=6, name='Frank', score=70.0, major='Physics'}
Student{id=4, name='David', score=59.0, major='Computer Science'}
Student{id=2, name='Bob', score=45.0, major='Mathematics'}

Updated consequence:
Updated: Student{id=5, name='Eve', score=95.0, major='Mathematics'}
Student with ID 100 not found.

Global Statistics:
Highest Scoring Student: Student{id=5, name='Eve', score=95.0, major='Mathematics'}
Number of Passed Students: 5
Number of Failed Students: 2
```

三、总结

本次作业通过运用 Java 集合框架（List、HashSet、HashMap）及自定义 Comparator，实现了学生数据的去重、分组、排序、更新与统计功能。代码满足基础需求，但在动态分组和代码可读性方面仍有优化空间。通过此次作业，我加深了对集合框架高级应用及多条件排序的理解，提升了对 Java 语言基础的掌握。